

EC200U&EG91xU Series

QuecOpen Decoding

Development Guide

LTE Standard Module Series

Version: 1.1

Date: 2023-09-01

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-12-09	Chaos HUANG	Creation of the document
1.0	2022-01-20	Chaos HUANG	First official release
1.1	2023-09-01	Richard LIANG	Added applicable modules EG912U-GL and EG915U series.

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
1.1. Applicable Modules	7
2 Decoding APIs.....	8
2.1. Header File	8
2.2. API Overview	8
2.3. API Description.....	8
2.3.1. ql_qr_decoder_init.....	8
2.3.1.1. ql_errcode_decoder_e.....	9
2.3.2. ql_qr_image_decoder	9
2.3.3. ql_qr_get_decoder_result	10
2.3.3.1. ql_decoder_type_e	10
2.3.4. ql_destroy_decoder.....	11
2.3.5. ql_get_decoder_version.....	11
3 Development Example and Debugging	12
3.1. Development Example of App Decoding	12
3.1.1. Static Library Use for Decoding	12
3.1.2. Decoding Example	12
3.1.3. Example Auto-Start	13
3.2. Debugging of Decoding.....	14
3.2.1. Debugging Procedure	14
3.2.2. Debugging Log Information.....	14
4 Appendix References	16

Table Index

Table 1: Applicable Modules..... 7

Table 2: API Overview 8

Table 3: Related Documents 16

Table 4: Terms and Abbreviations 16

Figure Index

Figure 1: Static Library Linking Example	12
Figure 2: Entry Function of Decoding Example	13
Figure 3: Setting of Decoding Example Auto-Start	13
Figure 4: Ports in Device Manager	14
Figure 5: Log Indicating Failure to Recognize QR Code or Barcode	14
Figure 6: Log Indicating QR Code or Barcode is Recognized Successfully	15

1 Introduction

Quectel EC200U series and EG91xU family modules support QuecOpen[®] solution. QuecOpen[®] is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen[®], see **document [1]**.

This document introduces decoding APIs, the relevant example and debugging of Quectel EC200U series and EG91xU family modules in QuecOpen[®] solution.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 Decoding APIs

2.1. Header File

`ql_api_decoder.h`, the header file of decoding APIs, is in the `components\ql-kernel\inc` directory.

2.2. API Overview

Table 2: API Overview

Function	Description
<code>ql_qr_decoder_init()</code>	Initializes decoding.
<code>ql_qr_image_decoder()</code>	Decodes the inputted barcode or QR code.
<code>ql_qr_get_decoder_result()</code>	Gets the decoding result.
<code>ql_destroy_decoder()</code>	Releases decoding.
<code>ql_get_decoder_version()</code>	Gets decoding library version.

2.3. API Description

2.3.1. `ql_qr_decoder_init`

This function initializes decoding.

- **Prototype**

```
ql_errcode_decoder_e ql_qr_decoder_init(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.1.1. ql_errcode_decoder_e

The enumeration of execution results:

```
typedef enum
{
    QL_DECODER_SUCCESS = QL_SUCCESS,
    QL_DECODER_INIT_ERR           = 1|QL_COMPONENT_BSP_DECODER,
    QL_DECODER_ERR,
    QL_DECODER_GET_RESULT_ERR,
    QL_DECODER_GET_RESULT_LENGTH_ERR,
    QL_DECODER_DESTROY_ERR,
}ql_errcode_decoder_e;
```

- **Member**

Member	Description
QL_DECODER_SUCCESS	Successful execution.
QL_DECODER_INIT_ERR	Failed to initialize decoding.
QL_DECODER_ERR	Failed to decode.
QL_DECODER_GET_RESULT_ERR	Failed to get the decoding results.
QL_DECODER_GET_RESULT_LENGTH_ERR	Failed to get the length of decoding results.
QL_DECODER_DESTROY_ERR	Failed to release decoding.

2.3.2. ql_qr_image_decoder

This function decodes the inputted barcode or QR code.

- **Prototype**

```
ql_errcode_decoder_e ql_qr_image_decoder(uint16_t *img_buffer, uint32_t width, uint32_t height)
```

- **Parameter**

**img_buffer:*

[In] Inputted images.

width:

[In] Width of inputted image.

height:

[In] Height of inputted image.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.3. ql_qr_get_decoder_result

This function gets the decoding result.

- **Prototype**

```
ql_errcode_decoder_e ql_qr_get_decoder_result (ql_decoder_type_e* type, unsigned char * result);
```

- **Parameter**

**type:*

[Out] Type of decoding. See **Chapter 2.3.3.1** for details.

**result:*

[Out] Buffer for storing the result of scanning.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.3.1. ql_decoder_type_e

The enumeration of decoding types:

```
typedef enum
{
    QL_DECODER_TYPE_CODE39 = 0,
    QL_DECODER_TYPE_CODE128,
    QL_DECODER_TYPE_QR_CODE,
    QL_DECODER_TYPE_NONE = 0xff,
```

```
}ql_decoder_type_e;
```

- **Member**

Member	Description
<code>QL_DECODER_TYPE_CODE39</code>	Code-39.
<code>QL_DECODER_TYPE_CODE128</code>	Code-128.
<code>QL_DECODER_TYPE_QR_CODE</code>	QR code.
<code>QL_DECODER_TYPE_NONE</code>	No recognition.

2.3.4. ql_destroy_decoder

This function releases decoding and releases the memory space requested by the decoder.

- **Prototype**

```
ql_errcode_decoder_e ql_destroy_decoder(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.5. ql_get_decoder_version

This function gets the current version of the decoding library.

- **Prototype**

```
ql_errcode_decoder_e ql_get_decoder_version(unsigned char* version);
```

- **Parameter**

**version:*

[Out] The current version of decoding library.

- **Return Value**

See **Chapter 2.3.1.1** for details.

3 Development Example and Debugging

3.1. Development Example of App Decoding

3.1.1. Static Library Use for Decoding

The module decoding library is used as a static library, therefore you need to link the static library when using it. The static library address is stored in *components/newlib/armca5/libq_l_api_decoder.a*. You only need to link the static library where the library is required. Please see *CmakeLists.txt* in the root directory of App SDK for details, as shown in the following figure:

```

if(CONFIG_APPING_LOAD_FLASH)
    set(target ${QL_APP_BUILD_VER})
    add_appimg_flash_ql_example(${target} ql_init.c)
    target_link_libraries(${target} PRIVATE ql_app_nw ql_app_peripheral ql_app_osi ql_app_dev ql_app_sim ql_app_power)
endif()
if(QL_APP_FEATURE_DECODER)
    add_library(ql_decoder_api STATIC IMPORTED)
    set_target_properties(ql_decoder_api PROPERTIES IMPORTED_LOCATION ${SOURCE_TOP_DIR}/components/newlib/armca5/libq_l_api_decoder.a)
    target_link_libraries(${target} PRIVATE ql_app_decoder ql_decoder_api ${libm_file_name})
endif()
if(QL_APP_FEATURE_FTP)
    target_link_libraries(${target} PRIVATE ql_app_ftp)
endif()
if(QL_APP_FEATURE_HTTP)
    target_link_libraries(${target} PRIVATE ql_app_http)
endif()

```

The App using decoding library Link static library

Figure 1: Static Library Linking Example

3.1.2. Decoding Example

decoder_demo.c, the decoding example file, is provided in the SDK of Quectel EC200U series and EG91xU family modules, in the *components/ql-application/decoder* directory.

The example mainly includes:

- Initializing decoding
- Scanning codes with camera
- Decoding
- Getting decoding results

The example entry function is *ql_decoder_app_init()*, as shown below:

```

83: void ql_decoder_app_init(void)
84: {
85:     QIDECODERStatus err = QL_OSI_SUCCESS;
86:     ql_task_t decoder_task = NULL;
87:
88:     err = ql_rtos_task_create(&decoder_task, QL_DECODER_TASK_STACK_SIZE, QL_DECODER_TASK_Prio, "decoder DEMO", ql_decoder_demo_thread, NULL, QL_DECODER_TASK_EVENT_CNT);
89:     if (err != QL_OSI_SUCCESS)
90:     {
91:         QL_DECODER_LOG("DECODER demo task created failed");
92:     }
93: }
94:

```

Figure 2: Entry Function of Decoding Example

In the example, decoding is initialized first, then the camera and LCD are turned on with preview enabled, and then code scanning, decoding and getting results are started in sequence.

3.1.3. Example Auto-Start

The above example is not started by default in *ql_init_demo_thread()*, as shown below. If necessary, uncomment *ql_decoder_app_init()* to start the example.

```

static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);
    #if 0
        ql_gpio_app_init();
        ql_gpioint_app_init();
    #endif

    #ifdef QL_APP_FEATURE_DECODER
        //ql_decoder_app_init();
    #endif

    #ifdef QL_APP_FEATURE_LEDCFG
        ql_ledcfg_app_init();
    #endif

    #ifdef QL_APP_FEATURE_AUDIO
        //ql_audio_app_init();
    #endif

    #ifdef QL_APP_FEATURE_LCD
        //ql_lcd_app_init();
    #endif
    #ifdef QL_APP_FEATURE_LVGL
        //ql_lvgl_app_init();
    #endif
}

```

Figure 3: Setting of Decoding Example Auto-Start

NOTE

The camera is required in decoding, so you need to uncomment *ql_ledcfg_app_init()* when you use the example. And some functionality pins required by the camera are also used in *ql_ledcfg_app_init()*, resulting in conflicts and abnormal camera operation. In addition, when using this example, do not execute *ql_camera_app_init()* or *ql_i2c_demo_init()* at the same time, because all three examples use cameras for demonstration purposes. If all three entry functions are started at the same time, they will conflict with each other and cause module exceptions.

3.2. Debugging of Decoding

3.2.1. Debugging Procedure

The decoding can be debugged on Quectel LTE OPEN EVB. The debugging procedure is as follows:

Download the compiled firmware to the module. Connect the USB port of the LTE OPEN EVB to a PC with a USB cable. The USB AP Log Port mainly displays the system debugging information. You can view the information of the example through USB AP Log Port after capturing the log with cooltools tool. For details about log capture, see **document [2]**.

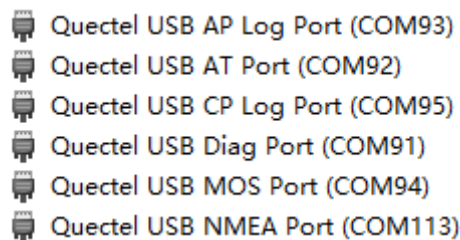


Figure 4: Ports in Device Manager

3.2.2. Debugging Log Information

Uncomment `ql_decoder_app_init()` so that the decoding example would start automatically at bootup and judge if the decoding runs successfully in the log information. If the decoding is successful, the QR code or barcode can be recognized. Otherwise, the QR code or barcode cannot be recognized.

The debugging information on AP log port is shown in **Figure 5** and **Figure 6**.

[ql_api_qr_code][ql_qr_decoder_init, 131] Qr code Initial success
[ql_api_qr_code][ql_qr_image_decoder, 161] decoder into preview data
[ql_api_qr_code][ql_qr_image_decoder, 179] *** Decoding end***
[ql_api_qr_code][ql_qr_image_decoder, 182] Imgbuffer Decoder failed
[ql_DECODERDEMO][ql_decoder_demo_thread, 98] QR code decode failed
[ql_api_qr_code][ql_qr_image_decoder, 161] decoder into preview data
[ql_api_qr_code][ql_qr_image_decoder, 179] *** Decoding end***
[ql_api_qr_code][ql_qr_image_decoder, 182] Imgbuffer Decoder failed
[ql_DECODERDEMO][ql_decoder_demo_thread, 98] QR code decode failed

Figure 5: Log Indicating Failure to Recognize QR Code or Barcode

```
[ql_api_qr_code][ql_qr_image_decoder, 161] decoder into preview data
[ql_api_qr_code][ql_qr_image_decoder, 179] *** Decoding end***
[ql_api_qr_code][ql_qr_image_decoder, 206] imgbuffer Decoder success
[ql_api_qr_code][ql_qr_get_decoder_result, 247] Get Result Length success
[ql_DECODERDEMO][ql_decoder_demo_thread, 90] Get QR code decode result **34561561513131651351351303531351351351313**
[ql_api_qr_code][ql_qr_image_decoder, 161] decoder into preview data
[ql_api_qr_code][ql_qr_image_decoder, 179] *** Decoding end***
[ql_api_qr_code][ql_qr_image_decoder, 206] imgbuffer Decoder success
[ql_api_qr_code][ql_qr_get_decoder_result, 247] Get Result Length success
[ql_DECODERDEMO][ql_decoder_demo_thread, 90] Get QR code decode result **34561561513131651351351303531351351351313**
```

Figure 6: Log Indicating QR Code or Barcode is Recognized Successfully

4 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen_CSDK_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen_Log_Capture_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
APP	Application
COM	Communication
EVB	Evaluation Board
IoT	Internet of Things
LCD	Liquid Crystal Display
LTE	Long-Term Evolution
PC	Personal Computer
RTOS	Real-Time Operating System
SDK	Software Development Kit
USB	Universal Serial Bus